

Exam2021

June 15, 2026

Denne skabelon indeholder fire sektioner: **Opgave 1** til **Opgave 4**. Hver sektion har fire under-spørgsmål: **Spørgsmål 1** til **Spørgsmål 4**. En notebook konverteres til pdf ved at køre: `jupyter nbconvert -to pdf ExamTemplate2026.ipynb` i terminalen. Når notebooken eksporteres/printes til PDF, starter hver ny opgave på en ny side.

0.1 Kopiérbar blok til indsættelse af billeder

0.1.1 Markdown-metode

Kopiér linjen nedenfor til en Markdown-celle og ret filnavn/størrelse efter behov:

```

```

0.1.2 Python-metode

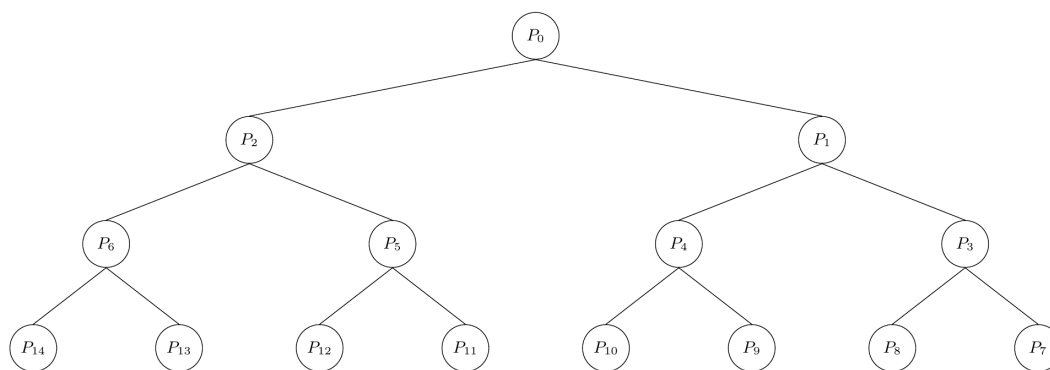
Kør kodecellen nedenfor, og brug funktionen `indsæt_billede(...)` i dine svar.

```
[79]: from IPython.display import Image, display

def indsæt_billede(sti, bredde=600):
    """Viser et billede i notebooken.

    Parametre:
    sti: Filsti til billedet, fx 'images/figur1.png'
    bredde: Billedets bredde i pixels, fx 600
    """
    display(Image(filename=sti, width=bredde))

indsæt_billede('tree.png', bredde=600)
```



```
[80]: # Imports
import numpy as np
import pulp as PLP
```

1 Opgave 1

Skriv din besvarelse nedenfor.

```
[81]: from operator import eq

import numpy as np

class LPFeasibleRegionPlotter:
    def __init__(
        self,
        coef_matrix,
        rhs,
        equality_types,
        x1_bounds=(0, 20),
        x2_bounds=(0, 20),
        grid_points=400,
        tol=1e-9,
    ):

        """
        Plot feasible regions for 2-variable LP constraints.

        Args:
            coef_matrix: Iterable of [a, b] for each constraint  $a*x_1 + b*x_2$ 
            ↪ (shape: n x 2)
            rhs: Iterable of right-hand side values c (length: n)
            equality_types: Iterable of constraint types: "<=", "<", ">=", ">",
            ↪ "=", "="
            x1_bounds: (min, max) for x1 axis
            x2_bounds: (min, max) for x2 axis
            grid_points: Resolution for feasible-region raster
            tol: Numerical tolerance
        """
        self.coef_matrix = np.asarray(coef_matrix, dtype=float) if coef_matrix
        ↪ is not None else None
        self.rhs = np.asarray(rhs, dtype=float) if rhs is not None else None
        self.equality_types = list(equality_types) if equality_types is not
        ↪ None else None
        self.x1_bounds = tuple(x1_bounds)
        self.x2_bounds = tuple(x2_bounds)
        self.grid_points = int(grid_points)
        self.tol = float(tol)
        self.objective_coeffs = None
        self.objective_value = None
```

```

def constraints_from_model(self, model):
    import pulp as PLP
    constraints = list(model.constraints.values())
    variables = model.variables()

    def format_sense(x):
        if x == 0:
            return "=="
        elif x == -1:
            return "<="
        elif x == 1:
            return ">="
        else:
            raise ValueError(f"Unknown constraint sense: {x}")

    coef_matrix = []
    rhs = []
    equality_types = []

    for constraint in constraints:
        rhs.append(-constraint.constant)
        equality_types.append(format_sense(constraint.sense))

        coef_matrix.append([
            constraint.expr.get(v, 0)
            for v in variables
        ])

    self.coef_matrix = np.array(coef_matrix)
    self.rhs = rhs
    self.equality_types = equality_types
    self.objective_coeffs = [model.objective.get(v, 0) for v in variables]
    model.solve()
    self.objective_value = PLP.value(model.objective)

def _validate_inputs(self):
    if self.coef_matrix.ndim != 2 or self.coef_matrix.shape[1] != 2:
        raise ValueError("coef_matrix must have shape (n_constraints, 2).")

    n = self.coef_matrix.shape[0]
    if len(self.rhs) != n or len(self.equality_types) != n:
        raise ValueError(
            "Lengths must match: len(coef_matrix) == len(rhs) ==_
↪len(equality_types).")

```

```

    )

    valid_eq = {"<=", "<", ">=", ">", "==", "="}
    bad = [e for e in self.equality_types if e not in valid_eq]
    if bad:
        raise ValueError(f"Invalid equality_types found: {bad}. Allowed:␣
↪{sorted(valid_eq)}")

    if self.x1_bounds[0] >= self.x1_bounds[1]:
        raise ValueError("x1_bounds must satisfy min < max.")
    if self.x2_bounds[0] >= self.x2_bounds[1]:
        raise ValueError("x2_bounds must satisfy min < max.")
    if self.grid_points < 50:
        raise ValueError("grid_points should be at least 50 for decent␣
↪visualization.")

    @staticmethod
    def _format_constraint_label(a, b, eq_type, c):
        def fmt_num(v):
            if abs(v - round(v)) < 1e-12:
                return str(int(round(v)))
            return f"{v:.3g}"

        parts = []
        if abs(a) > 1e-12:
            parts.append(f"{fmt_num(a)}$x_1$")
        if abs(b) > 1e-12:
            sign = "+" if b >= 0 and parts else ""
            parts.append(f"{sign}{fmt_num(b)}$x_2$")
        if not parts:
            lhs = "0"
        else:
            lhs = " ".join(parts)

        return f"{lhs} {eq_type} {fmt_num(c)}"

    def _constraint_line(self, a, b, c, x_vals):
        """
        Returns x2 values for line  $a*x_1 + b*x_2 = c$  if non-vertical.
        For vertical lines, caller should use  $x = c/a$ .
        """
        return (c - a * x_vals) / b

    def plot(
        self,
        objective_coeffs=None,
        optimal_value=None,

```

```

        show_constraints=True,
        show_feasible_region=True,
        show_objective=True,
        show_equality_bands=False,
        equality_band_tol=None,
        ax=None,
        title="Feasible Region",
        legend=True,
        show = True
    ):
        import matplotlib.pyplot as plt
        """
        Create the LP feasible region plot and return (fig, ax) for
        ↪post-editing.

        Args:
            objective_coeffs: (c1, c2) for objective  $c1*x1 + c2*x2$ 
            optimal_value: scalar Z value to plot objective level set  $c1*x1 +$ 
            ↪ $c2*x2 = Z$ 
            show_constraints: draw constraint boundaries
            show_feasible_region: shade feasible region
            show_objective: draw objective level-set line if objective data
            ↪provided
            show_equality_bands: if True, equality constraints also affect mask
            ↪via thin tolerance band
            equality_band_tol: tolerance for equality in mask (default derived
            ↪from grid spacing)
            ax: optional Matplotlib axis to draw on
            title: plot title
            legend: whether to show legend

        Returns:
            (fig, ax): Matplotlib figure and axis for further editing.
        """
        self._validate_inputs()
        # Prepare axis
        if ax is None:
            fig, ax = plt.subplots(figsize=(8, 8))
        else:
            fig = ax.figure
        if objective_coeffs is None and self.objective_coeffs is not None:
            objective_coeffs = self.objective_coeffs
        if optimal_value is None and self.objective_value is not None:
            optimal_value = self.objective_value

        x_vals = np.linspace(self.x1_bounds[0], self.x1_bounds[1], self.
        ↪grid_points)

```

```

        y_vals = np.linspace(self.x2_bounds[0], self.x2_bounds[1], self.
↪grid_points)
        x1_grid, x2_grid = np.meshgrid(x_vals, y_vals)

        feasible_mask = np.ones_like(x1_grid, dtype=bool)

        # Tolerance band for equality mask if requested
        dx = (self.x1_bounds[1] - self.x1_bounds[0]) / max(self.grid_points -
↪1, 1)
        dy = (self.x2_bounds[1] - self.x2_bounds[0]) / max(self.grid_points -
↪1, 1)
        eq_tol = equality_band_tol if equality_band_tol is not None else
↪max(dx, dy) * 1.25

        cmap = plt.get_cmap("tab10")
        used_labels = set()

        for i, (coef, c, eq_type) in enumerate(zip(self.coef_matrix, self.rhs,
↪self.equality_types)):
            a, b = coef
            lhs = a * x1_grid + b * x2_grid

            # Feasible mask update
            if eq_type in ("<=", "<"):
                feasible_mask &= lhs <= c + self.tol
            elif eq_type in (">=", ">"):
                feasible_mask &= lhs >= c - self.tol
            elif eq_type in ("==", "="):
                # Usually equalities define boundaries (not areas).
                # Optional banding can be enabled for visualization.
                if show_equality_bands:
                    feasible_mask &= np.abs(lhs - c) <= eq_tol

            # Plot constraint boundary
            if show_constraints:
                color = cmap(i % 10)
                label = self._format_constraint_label(a, b, eq_type, c)
                label = label if label not in used_labels else None
                if label:
                    used_labels.add(label)

            # Degenerate: 0*x1 + 0*x2 = c
            if abs(a) <= self.tol and abs(b) <= self.tol:
                # Skip plotting, but keep logical impact already handled
                continue

            if abs(b) > self.tol:

```

```

        y_line = self._constraint_line(a, b, c, x_vals)
        ax.plot(x_vals, y_line, color=color, linewidth=2,
↪label=label)
    else:
        # Vertical line:  $x = c / a$ 
        x_const = c / a
        ax.axvline(x=x_const, color=color, linewidth=2, label=label)

# Plot feasible region mask
if show_feasible_region:
    ax.imshow(
        feasible_mask.astype(float),
        extent=(
            self.x1_bounds[0],
            self.x1_bounds[1],
            self.x2_bounds[0], # correct y-min
            self.x2_bounds[1], # correct y-max
        ),
        origin="lower",
        cmap="Greens",
        alpha=0.30,
        interpolation="nearest",
        aspect="auto",
        zorder=0,
    )

# Objective level set line
if (
    show_objective
    and objective_coeffs is not None
    and optimal_value is not None
):
    c1, c2 = map(float, objective_coeffs)

    if abs(c1) <= self.tol and abs(c2) <= self.tol:
        pass # Degenerate objective; nothing to draw
    elif abs(c2) > self.tol:
        y_obj = (float(optimal_value) - c1 * x_vals) / c2
        ax.plot(
            x_vals,
            y_obj,
            "r--",
            linewidth=2,
            label=f"Objective level:  $Z = \{optimal\_value\}$ ",
        )
    else:
        x_obj = float(optimal_value) / c1

```



```

        ax.axvline(
            x=x_obj,
            color="r",
            linestyle="--",
            linewidth=2,
            label=f"Objective level: Z = {optimal_value}",
        )

    # Styling
    ax.set_xticks(range(int(self.x1_bounds[0]), int(self.x1_bounds[1]) + 1))
    ax.set_yticks(range(int(self.x2_bounds[0]), int(self.x2_bounds[1]) + 1))
    ax.grid(True)
    ax.set_xlim(self.x1_bounds)
    ax.set_ylim(self.x2_bounds)
    ax.set_xlabel("$x_1$")
    ax.set_ylabel("$x_2$")
    ax.set_title(title)
    ax.grid(True, linestyle="--", alpha=0.6)

    if legend:
        ax.legend(loc="best", fontsize=9)

    fig.tight_layout()
    if show:
        plt.show()
    return fig, ax

```

1.1 Spørgsmål 1

I overstående kode har jeg implementeret en klasse der plotter den brugbare region. `constraints_from_model` metoden gør at jeg kan definere en LP og få det brugbare område

```

[82]: x = PLP.LpVariable.dicts("x", indices= range(2), lowBound= 0)

model = PLP.LpProblem(
    name="Model",
    sense=PLP.LpMaximize)

# Objective
model += 8 * x[0] + 7*x[1]
# Constraints
model += 3*x[0] + 8*x[1] <= 48
model += 7*x[0] + 8*x[1] <= 56
model += 5*x[0] + 3*x[1] <= 30
model += 4*x[0] + 1*x[1] <= 22

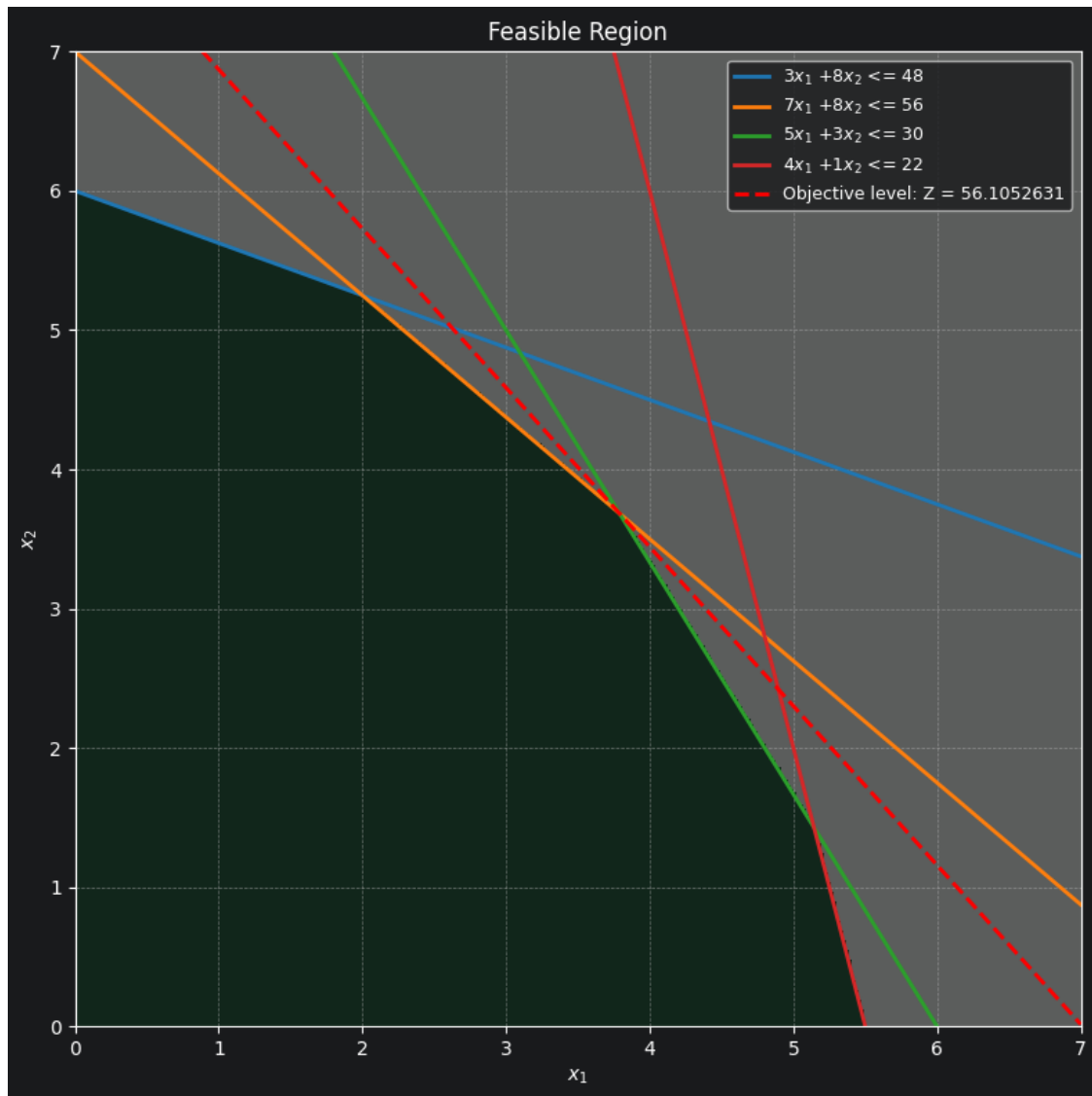
feas = LPFeasibleRegionPlotter(None,

```

```

None,
None,
x1_bounds=(0,7),
x2_bounds=(0,7),
)
feas.constraints_from_model(model)
feas.plot()

```



```

[82]: (<Figure size 800x800 with 1 Axes>,
      <Axes: title={'center': 'Feasible Region'}, xlabel='$x_1$', ylabel='$x_2$'>))

```

1.2 Spørgsmål 2

Problemet defineret i spørgsmål 1 løses

```
[83]: model.solve()
print("Objective value:", PLP.value(model.objective))
for v in model.variables():
    print(v.name, ":", v.varValue)
```

Objective value: 56.1052631

x_0 : 3.7894737

x_1 : 3.6842105

1.3 Spørgsmål 3

Jeg finder skyggepriserne

```
[84]: # Beregninger/kode til dette spørgsmål kan skrives her
# Duals (shadow prices)
# These can be interpreted as the profit we would gain/lose for a small
# relaxation/restriction in the constraint
for name, constraint in model.constraints.items():
    print(name, "dual =", constraint.pi)
```

_C1 dual = -0.0

_C2 dual = 0.57894737

_C3 dual = 0.78947368

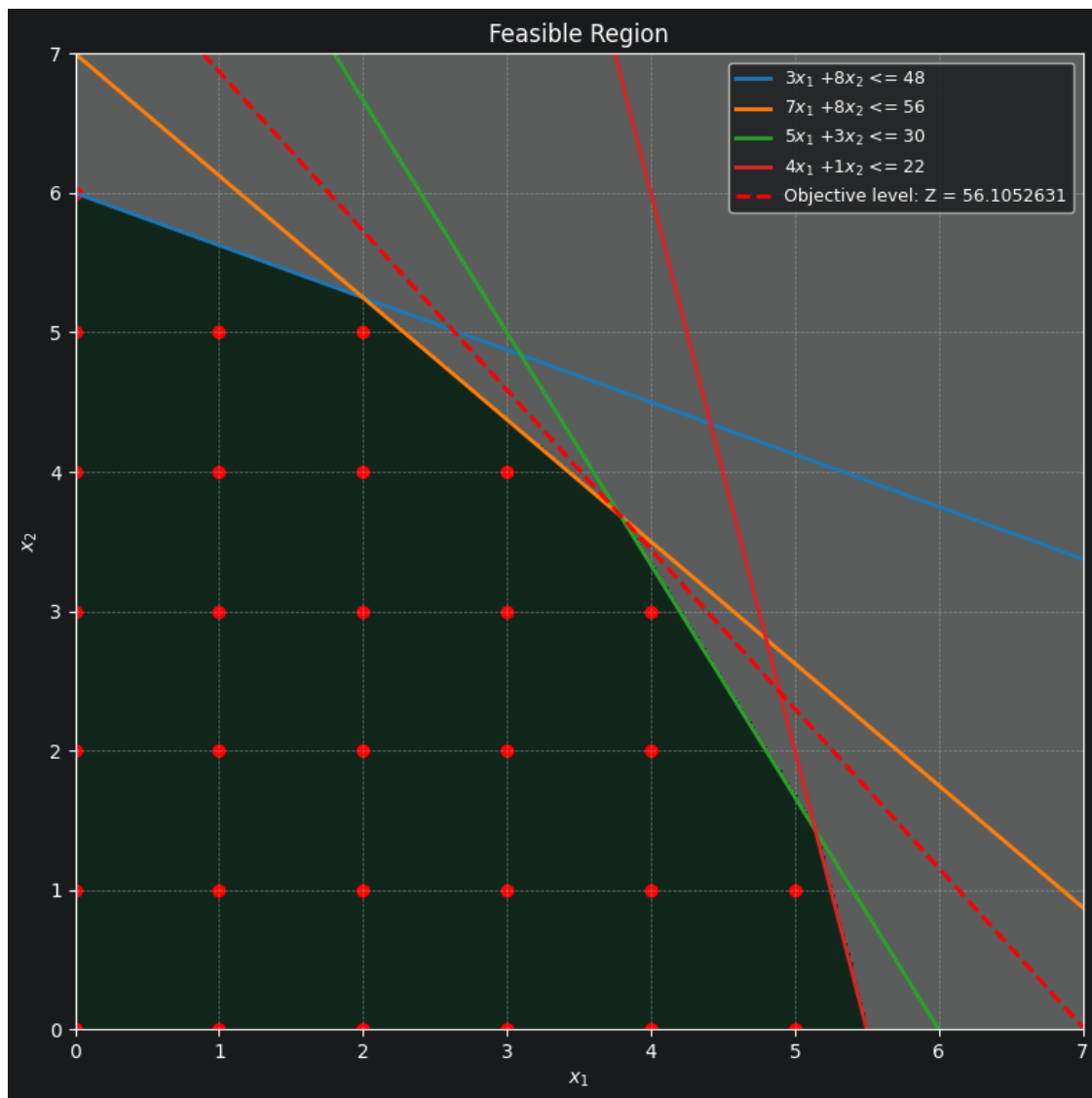
_C4 dual = -0.0

2 Opgave 2

2.1 Spørgsmål 1

De brugbare heltalspunkter markeres med en rød prik

```
[85]: feas.plot(show=False)
import matplotlib.pyplot as plt
points = [(0,x2) for x2 in range(7)]
points = points + [(1,x2) for x2 in range(6)]
points = points + [(2,x2) for x2 in range(6)]
points = points + [(3,x2) for x2 in range(5)]
points = points + [(4,x2) for x2 in range(4)]
points = points + [(5,x2) for x2 in range(2)]
x,y = zip(*points)
plt.scatter(x,y, c= "red")
plt.show()
```



2.2 Spørgsmål 2

Jeg har implementeret en rekursiv solver der benytter branch and bound til at løse ILP problemer ved hjælp af PULP LP problemer.

```
[102]: def branch_and_bound(
    LPmodel,
    sense,
    best_so_far=[None],
    objectives=[None],
    problem=[0],
    depth=0,
    branch_label="Root"
```

```

):
    import copy

    """
    :param LPmodel:
        input a Linear Programming relaxation of the ILP problem.
        Note it is important to keep track of if it is a minimization or
        ↪ maximization problem,
        as this will determine the branching strategy and pruning strategy.
    :return:
        Iterative Branch and Bound solution to the ILP problem.
    """

    indent = "    " * depth
    node_prefix = indent + ("    " if depth > 0 else "")

    print(f"{node_prefix}Problem {problem[0]}: {branch_label}")

    if best_so_far[0] is None:
        if sense == "maximize":
            best_so_far[0] = -10 ** 6
        else:
            best_so_far[0] = 10 ** 6

    if sense != "maximize" and sense != "minimize":
        raise ValueError("sense must be either 'maximize' or 'minimize'")

    if sense == "maximize":
        def objective_is_not_better(obj):
            return obj < best_so_far[0]

    if sense == "minimize":
        def objective_is_not_better(obj):
            return obj > best_so_far[0]

    def is_integer_value():
        """
        :return:
            dict where keys are variable names and values are boolean values
            ↪ indicating whether variable is integer.
        """
        eps = 10**-4
        vars = LPmodel.variables()

        d = dict()

```

```

        # Assign boolean values to the decision variables based on whether they
        ↪are integer or not
        for var in vars:
            if abs(var.varValue - ceil(var.varValue)) > eps and abs(var.
            ↪varValue - floor(var.varValue)) > eps:
                d[var.name] = False
            else:
                d[var.name] = True

        return d

    ### Step 1 - Solve problem ###
    LPmodel.solve(PLP.PULP_CBC_CMD(msg=0))
    obj = PLP.value(LPmodel.objective)

    ### Step 2 - Branch on non-integer decision variable ###
    vars = is_integer_value()

    print(indent + "    " + f"LP relaxation objective: {obj}")

    if LPmodel.status == PLP.LpStatusInfeasible:
        print(indent + "    " + "Pruned: infeasible")
        print()
        return objectives

    if objective_is_not_better(obj):
        print(
            indent + "    "
            + f"Pruned: objective {obj} worse than best so far {best_so_far[0]}"
        )

        print(indent + "    " + "Decision variables:")
        for v in LPmodel.variables():
            print(indent + "        " + f"    {v.name} = {v.varValue}")

        print(indent + "    " + f"Objective value: {obj}")
        print()
        return objectives

    if all([v for v in vars.values()]):
        print(indent + "    " + "Integer feasible solution found")
        print(indent + "    " + "Decision variables:")

        for v in LPmodel.variables():
            print(indent + "        " + f"    {v.name} = {v.varValue}")

        print(indent + "    " + f"Objective value: {obj}")

```

```

    if obj > best_so_far[0] and sense == "maximize":
        best_so_far[0] = obj

    if obj < best_so_far[0] and sense == "minimize":
        best_so_far[0] = obj

    print(indent + "    " + f"Updated best so far: {best_so_far[0]}")
    print()

    objectives[0] = obj
    return objectives

for name, value in vars.items():
    if value:
        continue
    else:
        branch_name = name
        branch_value = LPmodel.variablesDict()[branch_name].varValue
        break

### Step 3 - Create two branches and solve recursively ###

print(indent + "    " + "Current LP solution:")
for v in LPmodel.variables():
    print(indent + "    " + f"    {v.name} = {v.varValue}")

print(indent + "    " + f"Branching on {branch_name} = {branch_value}")
print(indent + "    " + f"Best so far: {best_so_far[0]}")
print()

# Left branch
left_model = copy.deepcopy(LPmodel)
left_model += left_model.variablesDict()[branch_name] <= floor(branch_value)

problem[0] = problem[0] + 1

branch_and_bound(
    left_model,
    sense,
    best_so_far,
    objectives,
    problem,
    depth=depth + 1,
    branch_label=f"{branch_name} <= {floor(branch_value)}"
)

```



```

# Right branch
right_model = copy.deepcopy(LPmodel)
right_model += right_model.variablesDict()[branch_name] >=
↪ceil(branch_value)

problem[0] = problem[0] + 1

branch_and_bound(
    right_model,
    sense,
    best_so_far,
    objectives,
    problem,
    depth=depth + 1,
    branch_label=f"{branch_name} >= {ceil(branch_value)}"
)

return best_so_far[0]

```

Ovenstående kode benyttes til at løse problemet. (x1, x2) angives i parentes, objekt værdi som enestående tal.

```
[103]: branch_and_bound(LPmodel = model, sense="maximize")
```

Problem 0: Root

LP relaxation objective: 56.1052631

Current LP solution:

x_0 = 3.7894737

x_1 = 3.6842105

Branching on x_0 = 3.7894737

Best so far: -1000000

Problem 1: x_0 <= 3

LP relaxation objective: 54.625

Current LP solution:

x_0 = 3.0

x_1 = 4.375

Branching on x_1 = 4.375

Best so far: -1000000

Problem 2: x_1 <= 4

LP relaxation objective: 52.0

Integer feasible solution found

Decision variables:

x_0 = 3.0

x_1 = 4.0

Objective value: 52.0

Updated best so far: 52.0

Problem 3: $x_1 \geq 5$
LP relaxation objective: 53.2857144
Current LP solution:
 $x_0 = 2.2857143$
 $x_1 = 5.0$
Branching on $x_0 = 2.2857143$
Best so far: 52.0

Problem 4: $x_0 \leq 2$
LP relaxation objective: 52.75
Current LP solution:
 $x_0 = 2.0$
 $x_1 = 5.25$
Branching on $x_1 = 5.25$
Best so far: 52.0

Problem 5: $x_1 \leq 5$
LP relaxation objective: 51.0
Pruned: objective 51.0 worse than best so far 52.0
Decision variables:
 $x_0 = 2.0$
 $x_1 = 5.0$
Objective value: 51.0

Problem 6: $x_1 \geq 6$
LP relaxation objective: 42.0
Pruned: objective 42.0 worse than best so far 52.0
Decision variables:
 $x_0 = 0.0$
 $x_1 = 6.0$
Objective value: 42.0

Problem 7: $x_0 \geq 3$
LP relaxation objective: 58.125
Pruned: infeasible

Problem 8: $x_0 \geq 4$
LP relaxation objective: 55.333333100000004
Current LP solution:
 $x_0 = 4.0$
 $x_1 = 3.3333333$
Branching on $x_1 = 3.3333333$
Best so far: 52.0

Problem 9: $x_1 \leq 3$
LP relaxation objective: 54.6
Current LP solution:

```

x_0 = 4.2
x_1 = 3.0
Branching on x_0 = 4.2
Best so far: 52.0

Problem 10: x_0 <= 4
LP relaxation objective: 53.0
Integer feasible solution found
Decision variables:
  x_0 = 4.0
  x_1 = 3.0
Objective value: 53.0
Updated best so far: 53.0

Problem 11: x_0 >= 5
LP relaxation objective: 51.666666899999996
Pruned: objective 51.666666899999996 worse than best so far 53.0
Decision variables:
  x_0 = 5.0
  x_1 = 1.6666667
Objective value: 51.666666899999996

Problem 12: x_1 >= 4
LP relaxation objective: 56.1052631
Pruned: infeasible

```

[103]: 53.0

2.3 Spørgsmål 3

Indsæt din besvarelse her.

[88]: *# Beregninger/kode til dette spørgsmål kan skrives her*

2.4 Spørgsmål 4

Indsæt din besvarelse her.

[89]: *# Beregninger/kode til dette spørgsmål kan skrives her*

3 Opgave 3

Skriv din besvarelse nedenfor.

3.1 Spørgsmål 1

Indsæt din besvarelse her.

[90]: `# Beregninger/kode til dette spørgsmål kan skrives her`

3.2 Spørgsmål 2

Indsæt din besvarelse her.

[91]: `# Beregninger/kode til dette spørgsmål kan skrives her`

3.3 Spørgsmål 3

Indsæt din besvarelse her.

[]: `%%sql`

[92]: `# Beregninger/kode til dette spørgsmål kan skrives her`

3.4 Spørgsmål 4

Indsæt din besvarelse her.

[93]: `# Beregninger/kode til dette spørgsmål kan skrives her`

4 Opgave 4

Skriv din besvarelse nedenfor.

4.1 Spørgsmål 1

Indsæt din besvarelse her.

[94] : `# Beregninger/kode til dette spørgsmål kan skrives her`

4.2 Spørgsmål 2

Indsæt din besvarelse her.

[95] : `# Beregninger/kode til dette spørgsmål kan skrives her`

4.3 Spørgsmål 3

Indsæt din besvarelse her.

[96] : `# Beregninger/kode til dette spørgsmål kan skrives her`

4.4 Spørgsmål 4

Indsæt din besvarelse her.

[97] : `# Beregninger/kode til dette spørgsmål kan skrives her`